Module map: what to borrow for another project

English version of ../../modules/module-map.md.

The app/ package is assembled so that its parts can be carried away separately. The table answers "what to take if I need X": the minimal set of files, the dependencies, and what has to be replaced.

Need	Files to take	External dependencies	Replace
Sign-up, sign-in, cookie sessions	app/security.py, app/routers/auth.py, app/models.py	fastapi, pydantic	users/sessions tables from app/db.py or your own repository
Only passwords and tokens	app/security.py (hash_password, validate_password, create_session)	—	db.cursor with your own database access; for a new project use argon2/bcrypt instead of SHA-256
SQLite layer without an ORM	the whole app/db.py	—	SCHEMA and ADDED_COLUMNS for your tables
Sending e-mail with attachments	app/mailer.py	—	nothing: configuration arrives as SmtpSettings
Admin access by a code from the environment	app/security.py (check_admin_code, require_admin)	fastapi	the list of allowed roles
A validated form or request	app/routers/quiz.py (validate, normalize_phone)	fastapi	domain fields and messages
Configuration from the environment	app/config.py	—	the fields of Settings
HTML pages and an API in one process	app/routers/pages.py, server.py	fastapi, uvicorn	template and static paths
Optional keep-alive	app/keepalive.py	requests	URL and interval

Transfer rules

1. **One module — one meaning.** If a transfer needs "three more files", it is not a module but a system: take the whole thing.
2. **Dependencies are visible at the top.** A module imports only what the table lists; there is no hidden global state — configuration is read by settings(), not at import time.
3. Deliberately **flat**: no base classes and no plugin registry, because they cannot be moved without moving the whole hierarchy.
4. **Tests are part of the module.** When you take a module, take its tests: security → tests/test_auth.py, tests/test_security.py; quiz → tests/test_quiz.py.

5. **Backwards compatibility.** `server.py` keeps `app` and `init_db`, so existing `from server import app` imports keep working.

What is not here

- No `Repository` abstraction and no ORM: the project has one database and one consumer. For PostgreSQL projects see `docs/modules/` in ForgeRTKCRM (SQLAlchemy + Alembic).
- No plugin system and no DI container: FastAPI `Depends` covers the need (`security.require_admin`).